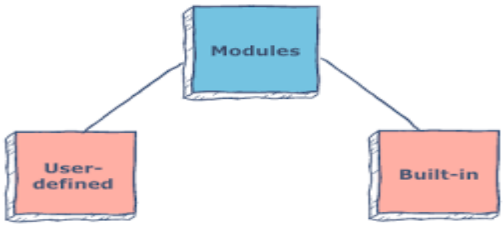
# Unit 5 Python Modules and Packages

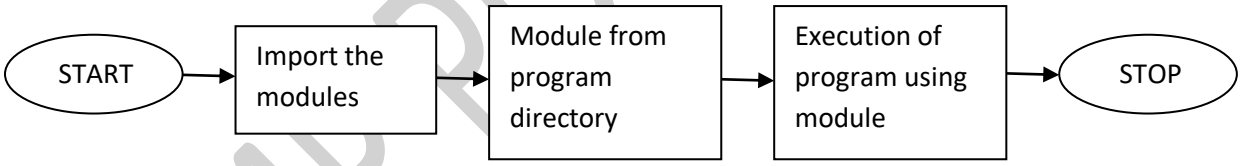
## ❖ Introduction to python module:

**What are modules in Python?**

- ✓ Definition: "Modules are predefined files that contain the python codes which include the basic functionality of class, methods ,variables etc."
- ✓ Modules can be divided in to two parts:




- ✓ Built-in modules: That are already programmed into the language
- ✓ User defined modules: That are created by user.
- ✓ We can define our most used functions in a module and import it.
- ✓ A file containing Python code, for example: example.py, is called a module, and its module name would be **example.**
- ✓ Example :OS, Math,Matplotlib
- ✓ Mechanism of python modules:




- ✓ Listing of modules: Command used for command prompt is >>>help("modules")
- ✓ It will show you the list of modules of python.

## ❖ Writing Module:

Creating a module is as simple as writing python code in .py file.

**Example of user defined module (module name: example):**

```
def add(a, b):
        result = a + b
        print( result)
```

- ✓ Here, we have defined a function add() inside a module named **example**. The function takes in two numbers and returns their sum.

VPMP POLYTECHNIC

- **Importing Module**

**How to import modules in Python?**

- ✓ We use the import keyword to do this. To import our previously defined module **example**, use this code:

  >>**import example**

- ✓ Using the module name we can access the function using the dot ( . ) operator. For example:

  >>> **example.add(4,5.5)**

  9.5

- ✓ Python has list of standard modules and their use cases. These files are in the **Lib** directory inside the location where you installed Python.

## ❖ Importing objects from module

Python provides two types of statements to use functionality of module:
1. import statement
2. from-import statement

Also we can import specific functions, classes or variables.
Example:

```
from example import add
print(add(2,3))

#we can also rename the imported object using the as keyword:
from example import add as sum
print(sum(2,3))
```

## ❖ Built-in Modules

Python Standard library is a collection of modules and packages.
Some common used Standard library modules are:

1. os: It interact with operating system.
   Ex: import os
   os.getcwd()
2. Sys: Access to system-specific parameters and functions.
   Ex: import sys
   print(sys.plateform)
3. Math: Performs mathematical functions and access constants.
   Ex: import math
   print(math.sqrt(16))

4. Datetime: Enables manipulation and formatting dates, times and time intervals.
   Ex: from datetime import datetime
   print(datetime.now())

VPMP POLYTECHNIC

5. Random: Generate random numbers, select random elements and shuffle sequences.
   Ex: import random
   print(random.randint(1,10))

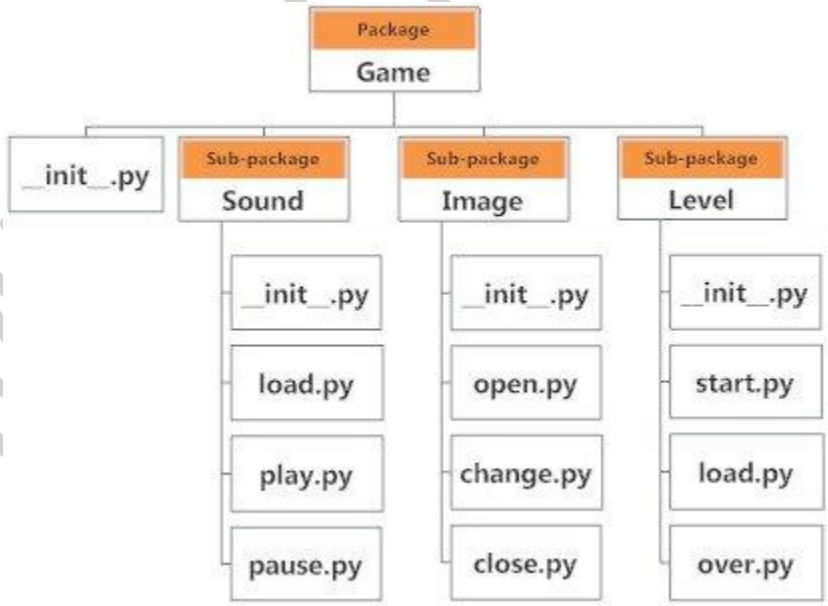
### ❖ Numeric and Mathematic module

Python provide modules like math and random for numeric and mathematical operations.
Math module:
Ex: import math
print(math.factorial(5)) #output: 120
Random module:
Import random
Print(random.randint(1,10)) #output: random no between 1 to 10

## • Introduction to Packages

**What are packages?**

- ✓ We don't usually store all of our files on our computer in the same location.
- ✓ We use a well-organized hierarchy of directories for easier access.
- ✓ **Python has packages for directories and modules for files.**
- ✓ **As a directory can contain subdirectories and files, a Python package can have sub-packages and modules.**
- ✓ Here is an example. Suppose we are developing a **game**. One possible organization of packages and modules could be as shown in the figure below.




- ✓ Defintion: A package is a collection of modules organized in directories with an __init__.py file.
- ✓ Package help organize code into namespaces. A directory must contain a file name __init__.py in order to consider it as a package.
- ✓ Ex: mypackage/
  __init__.py
  module1.py

module2.py

- **Writing a Package**

**Steps to create a package:**

1. Create a directory for the package.
2. Add an __init__.py file (it can be empty)
3. Add module files inside the package directory.

Example: Directory structure:

```
mypackage/
        __init__.py
        add.py
        sub.py
```

Content of add.py

```
def sum(a,b)
    return a+b
```

Content of sub.py

```
def minus(a,b)
    return a-b
```

Using the package:

```
from mypackage.add import sum
from mypackage.sub import minus
print(sum(10,20))    #output: 30
print(minus(20,10)) #output: 10
```

- **PIP-Package Installer for Python:**

✓ PIP is the package manager for Python packages.
✓ It is a package management system used to install and manage software packages written in Python.
✓ We use pip to install packages that do not come with Python.
✓ Basic syntax for pip commands in command prompt is: **pip 'argument'**

**Installing PIP:**

- Python pip comes pre-installed on 3.4 or older versions of python. To check pip is installed or not use command : pip –version
- It will display version of python which is installed.
- Packages/libraries written in python are stored in on-line repository known as Python Package Index(PyPI).
- PyPI is default source of packages and their dependencies.
- Use command: pip install package_name , PyPI will look for this package, download and install on your system.

VPMP POLYTECHNIC

**Download and install pip:**

1. Download the get-pip.py file and store it in same directory as python is installed.
2. Change the current path
3. Run the command : python get-pip.py
4. Verification of installation process is done
5. Once the installation has been correctly check the version using : pip -V

**Installing/Uninstalling python packages:**

1. Open the command line interface and tell PIP to download package you want.
2. Navigate the command line to location of script directory and type a command to install the package.
3. Ex: Download package name "mypack" then command on that directory is : pip install mypack
4. One can use package using import
5. To uninstall use command: pip uninstall mypack

**Difference between package and module**

| | Module | Package |
|---|---|---|
| 1 | There are set of predefined files. | It is a directory that holds modules |
| 2 | File contains basic functionalities of class, methods, variables etc. | Directory that holds modules and sub-packages. |
| 3 | To create a module use .py file with code | To create package, generate a directory |
| 4 | A file containing Python code, for example: example.py, is called a module, and its module name would be example. | Create a directory, Add an __init__.py file, Add module files inside the package directory. |
| 5 | Example :OS, Math,Matplotlib | Example: Directory structure:<br>mypackage/<br>__init__.py<br>add.py<br>sub.py |
| 6 | Module can be used using keyword **import** | Sub-packages can be used using keyword **from** in package. |

- **MATPLOTLIB Package:**

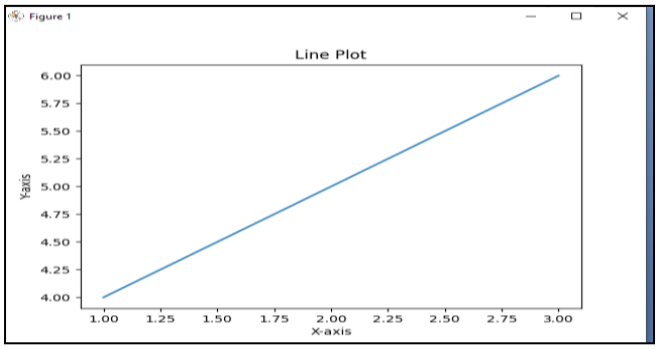
  It is python library for creating visualization.

  **Ex:**

```
#Line plot
import matplotlib.pyplot as plt
plt.plot([1,2,3],[4,5,6])
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Line Plot")
plt.show()
```

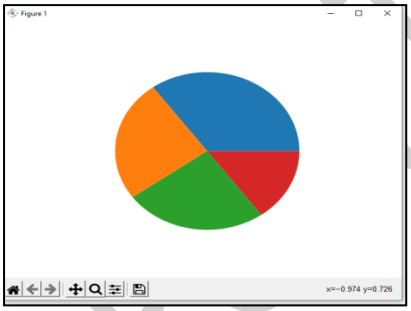
VPMP POLYTECHNIC

**Output:**




**Ex:**

```
#Pie plot
import matplotlib.pyplot as plt
import numpy as np
y=np.array([35,25,25,15])
plt.pie(y)
plt.show()
```




**Function overview:**

- plot(): create a line plot.
- show(): display plot.
- xlabel(),ylabel(): label the axes.
- grid(): Add a grid to plot.
- bar(): Create a bar chart.
- hist(): Create a histogram.
- subplot(): Adds multiple plots in one figure.
- pie(): Creates a pie chart.
- savefig(): Saves plot as an image file.
-

- **Numpy Package**

❖ **What is NumPy?**

- NumPy stands for Numerical Python.

- Numpy is a library for numerical computations in python.
- NumPy is a Python library used for working with arrays.
- It also has functions for working in domain of linear algebra, fourier transform, and matrices.
- NumPy was created in 2005 by Travis Oliphant. It is an open source project and you can use it freely.

**Creating Numpy arrays:**

```
import numpy as np
arr = np.array([1,2,3])
print(arr)  #output: [1 2 3 ]
```

**Array attributes:**

- shape: Dimensions of the array.
- ndim: Number of dimensions.
- size: Total number of elements.
- dtype: Datatype of elements.
- itemsize: Size of each element in bytes.
- Example:

```
arr = np.array([[1,2],[3,4]])
print(arr.shape)  #output: [2 2]
print(arr.ndim)  #output: 2
```

**Array Initialization:** Array will be initialized automatically with the use of Numpy.

- Example:

```
import numpy as np
z = np.zeros((2,2))
print("Initialization with zeros:\n",z)
x = np.ones((3,3))
print("Initialization with ones:\n",x)
y = np.eye(3)
print("Initialization with Identity:\n",y)
```

**Indexing and Slicing:** We can apply indexing and slicing using Numpy.

- Example:

```
import numpy as np
arr = np.array([1,2,3,4])
print(arr[1:3])  #output: [2 3]
```

**Array Operations:**

- Example:

```
import numpy as np
a1 = np.array([1,2,3])
a2 = np.array([9,10,11])
print(a1 + a2) #output: [10 12 14]
```

VPMP POLYTECHNIC

**Basic Math and Statistical Functions:**

- np.sum(): sum of array elements.
- np.min(), np.max(): Minimum and Maximum values.
- np.mean(): Mean value.
- np.std(), np.var() Standard deviation and variance.
- np.dot(), np.cross(), np.matmul(): Matrix operations.
- Example1:

```
arr = np.array([1,2,3,4])
print(np.sum(arr))    #10
print(np.mean(arr)) # 2.5
print(np.min(arr))    # 1
print(np.max(arr))  # 4
```

- Example2:

```
# numpy.std() method: Compute the standard deviation of the given data.
import numpy as np
# 1D array
arr = [2,4]
print("std of arr : ", np.std(arr)) #std of arr :  1.0
```

- Example3:

```
# numpy.var() method: Compute the variance of the given data.
import numpy as np
# 1D array
arr = [2,4]
print("std of arr : ", np.var(arr) #std of arr :  1.0
```

- Example4:

```
# numpy.dot() method: Returns the dot product of vectors a and b.
v1 = [1, 2]
v2 = [3, 4]
product = np.dot(v1,v2)
print("Dot Product:", product)  #Dot Product: 11
```

- Example5:

```
# numpy.cross() method: Find the cross product of two given vectors.
v1 = [1, 2]
v2 = [3, 4]
product = np.cross(v1,v2)
print("Cross Product:", product)  # Cross Product: -2
```

- Example6:

```
# numpy.matmul() method: performs matrix multiplication
v1 = [1, 2]
v2 = [3, 4]
product = np.matmul(v1,v2)
print("Matrix multiplication:", product) #:   Matrix multiplication: -2
```

VPMP POLYTECHNIC